38. Databases: SQL Drivers, Transactions, Migrations

38. Databases: SQL Drivers, Transactions, Migrations

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2 days
Slug: `38-databases-sql`

Learning objectives

- Use database/sql with a driver
- Wrap multi-step writes in transactions
- Think in migrations for schema change
- Avoid SQL injection with parameters

Prerequisites

Complete prior numbered lessons. Type every example yourself.

database/sql

```
db, err := sql.Open("pgx", dsn)
db.SetMaxOpenConns(20)
row := db.QueryRowContext(ctx, `SELECT name FROM users WHERE id=$1`, id)
```

Always use contexts and placeholders — never string-concatenate user input into SQL.

Transactions

```
tx, err := db.BeginTx(ctx, nil)
defer tx.Rollback()
// execs...
return tx.Commit()
```

Migrations

Treat schema as versioned code (golang-migrate, goose, atlas). Expand/contract for zero-downtime.

Common mistakes (especially from Python instincts)

- sql.Open without Ping
- Holding transactions open during HTTP calls to others
- SELECT * forever

Exercises

Work in `exercises/38-databases-sql/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/38-databases-sql/` until you have an honest failing attempt.

Self-check

1. Why placeholders?
2. What does Rollback on defer do after Commit?
3. Expand/contract meaning?

Further reading

- https://pkg.go.dev/database/sql